第零讲 算法组导论&Linux&git

> ITL战队算法组培训——2027秋季
> 导论与开发环境基础

> 本讲不是线下授课，而是作为正式培训前的导论与基础，请各位同学在正式培训之前完成本讲的学习

导论

什么是算法?

根据ISO/IEC 2382:2015 术语编号 2121376，算法（Algorithm）被定义为：

> A finite ordered set of well-defined rules for the solution of a problem.

即算法是为解决某一问题而规定的一组有限、有序且明确定义的规则。

这是一个非常宽泛的概念，而放在robomaster中：

- 如何让机器人从复杂的视觉信息中准确识别目标?
- 如何根据运动状态预测目标未来的位置?
- 如何使机器人感知自身所处的环境，并自主规划运动路径?
- 如何让机器人根据不断变化的赛场局势，做出合理的行动决策?

诸如此类问题，是算法在真实物理世界和robomaster赛场中的具体体现

将这些实际问题抽象为数学模型，设计相应的计算方法，再通过程序实现、仿真验证与实车调试，将算法转化为机器人实际能力的团队，就是我们算法组

算法组平时做什么?

当然，我们面对的也不是孤立的某一个算法\问题，而是一个从观测到行动的信息处理系统

传感器获得的数据通常带有噪声、延迟和不完整性。算法需要先理解这些数据，经过感知、坐标变换、状态估计和环境建模，形成相对可信的状态，再根据任务和约束生成动作或决策信息，最后通过通信接口交给其他模块或硬件执行

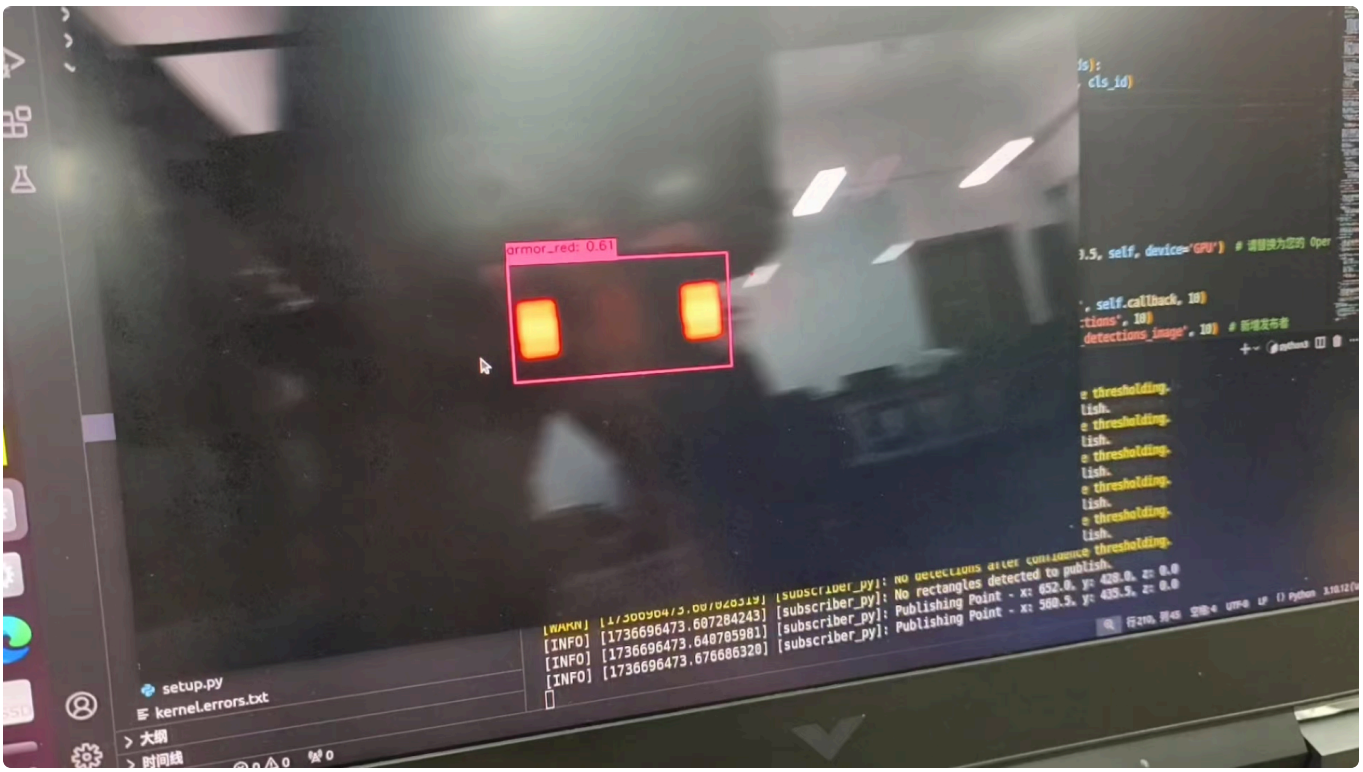
因此，算法组的共同工作可以概括为：
传感器观测
→ 信息提取

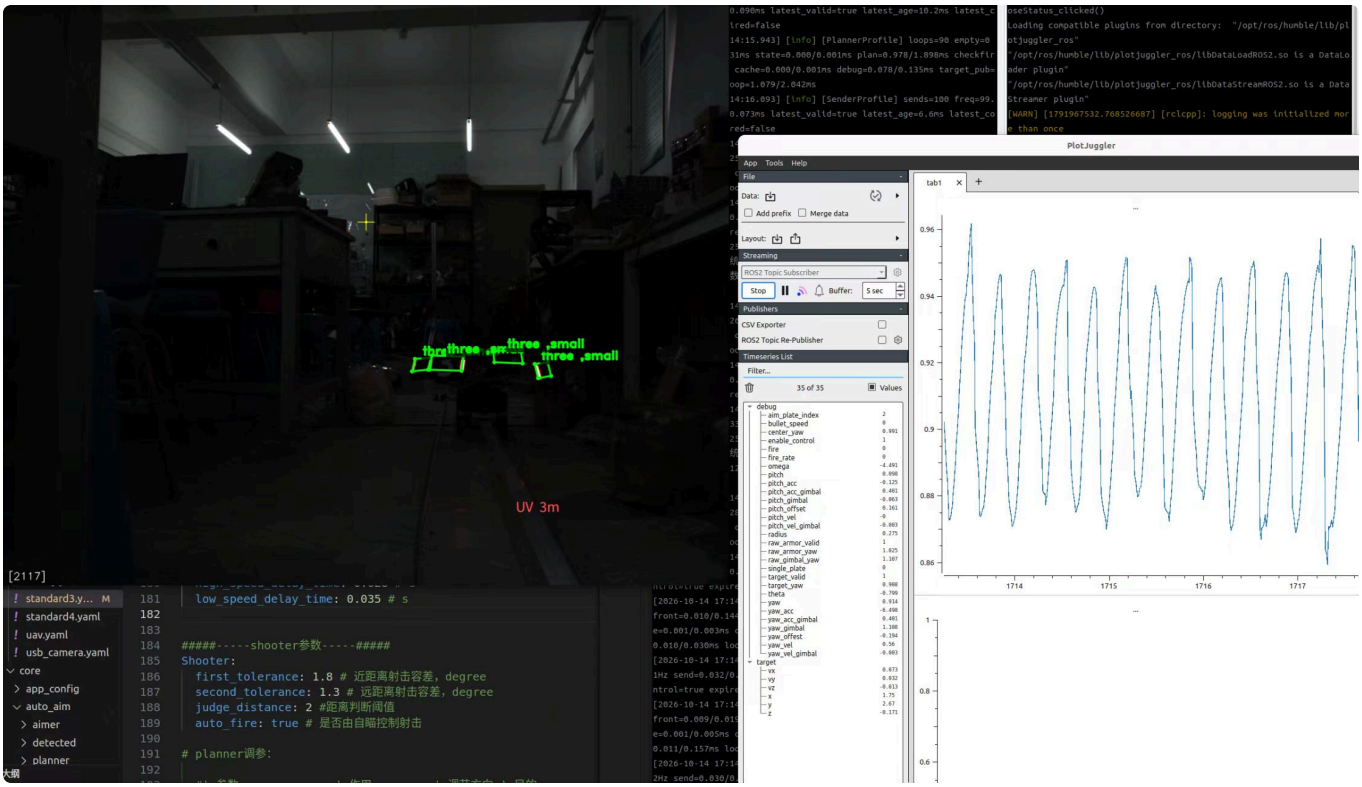
→ 坐标与时间统一
→ 状态估计和环境建模
→ 决策、规划或目标输出
→ 通信与硬件执行
→ 反馈、验证和故障处理
Linux、构建系统和 Git 则保证这条链路能够被开发、复现、协作和维护

算法组主要工作的主要工作有三个方向，而无论你将来从事哪个方向，都将学习到的共同技术栈：

- 开发环境与协作：Linux 命令行、文件与进程管理、设备与环境变量、编译构建、依赖管理、Git 版本控制和团队协作
- 软件工程与 C++/Python：模块划分、函数与类、数据结构、资源管理、并发、错误处理、配置管理，以及用 Python 进行数据分析和实验
- 机器人软件架构：将传感器、算法、决策和执行机构组织成相互协作的模块，理解节点、消息、服务、参数、启动文件和数据生命周期
- 数学、坐标与时间：向量和矩阵、坐标系、刚体变换、旋转表示、单位、时间戳、延迟和传感器同步
- 状态估计与不确定性：理解观测、状态、噪声、模型、预测和更新，知道算法输出为什么不能简单等同于真实状态
- 调试、验证与复现：日志、可视化、数据录制与回放、实验对照、性能测量、故障注入、版本记录和他人复现

自瞄






```
  1/100     21.1G      1.301       1.49      1.073        1208         640: 100%|██████████| 211/211 [00:58<00:00,  3.58it/s]
             Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100%|██████████| 12/12 [00:07<00:00,  1.71it/s]
               all        756      16610      0.634      0.214      0.206      0.112

  Epoch    GPU_mem   box_loss   cls_loss   df1_loss  Instances       Size
  2/100      24.2G      1.177     0.7263     0.9189       1038         640: 100%|██████████| 211/211 [00:46<00:00,  4.52it/s]
             Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100%|██████████| 12/12 [00:06<00:00,  1.94it/s]
               all        756      16610      0.595      0.328       0.31       0.16

  Epoch    GPU_mem   box_loss   cls_loss   df1_loss  Instances       Size
  3/100      24.2G       1.19     0.7003     0.9154       1211         640: 100%|██████████| 211/211 [00:46<00:00,  4.51it/s]
             Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100%|██████████| 12/12 [00:06<00:00,  1.81it/s]
               all        756      16610      0.492      0.378      0.353      0.173

  Epoch    GPU_mem   box_loss   cls_loss   df1_loss  Instances       Size
  4/100      24.2G      1.168     0.6687     0.9117        907         640: 100%|██████████| 211/211 [00:47<00:00,  4.47it/s]
             Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100%|██████████| 12/12 [00:06<00:00,  1.97it/s]
               all        756      16610      0.591      0.417      0.404      0.204

  Epoch    GPU_mem   box_loss   cls_loss   df1_loss  Instances       Size
  5/100      24.2G      1.134     0.6229     0.9032        969         640: 100%|██████████| 211/211 [00:46<00:00,  4.53it/s]
             Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100%|██████████| 12/12 [00:06<00:00,  1.80it/s]
               all        756      16610      0.602      0.441      0.435      0.223

  Epoch    GPU_mem   box_loss   cls_loss   df1_loss  Instances       Size
  6/100      24.2G      1.104     0.6004     0.8952       1118         640: 100%|██████████| 211/211 [00:46<00:00,  4.54it/s]
             Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100%|██████████| 12/12 [00:06<00:00,  1.97it/s]
               all        756      16610      0.655      0.469      0.466      0.239

  Epoch    GPU_mem   box_loss   cls_loss   df1_loss  Instances       Size
  7/100      24.2G      1.085     0.5847     0.8884       1215         640: 100%|██████████| 211/211 [00:46<00:00,  4.56it/s]
             Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100%|██████████| 12/12 [00:06<00:00,  1.83it/s]
               all        756      16610      0.647      0.487      0.484      0.254

  Epoch    GPU_mem   box_loss   cls_loss   df1_loss  Instances       Size
  8/100      24.2G      1.072     0.5687     0.8854       1058         640: 100%|██████████| 211/211 [00:47<00:00,  4.46it/s]
             Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100%|██████████| 12/12 [00:06<00:00,  1.99it/s]
               all        756      16610      0.657      0.522      0.508      0.273

  Epoch    GPU_mem   box_loss   cls_loss   df1_loss  Instances       Size
  9/100      24.2G      1.055     0.5611     0.8819       1159         640:  78%|███████▊  | 164/211 [00:37<00:12,  3.65it/s]
```

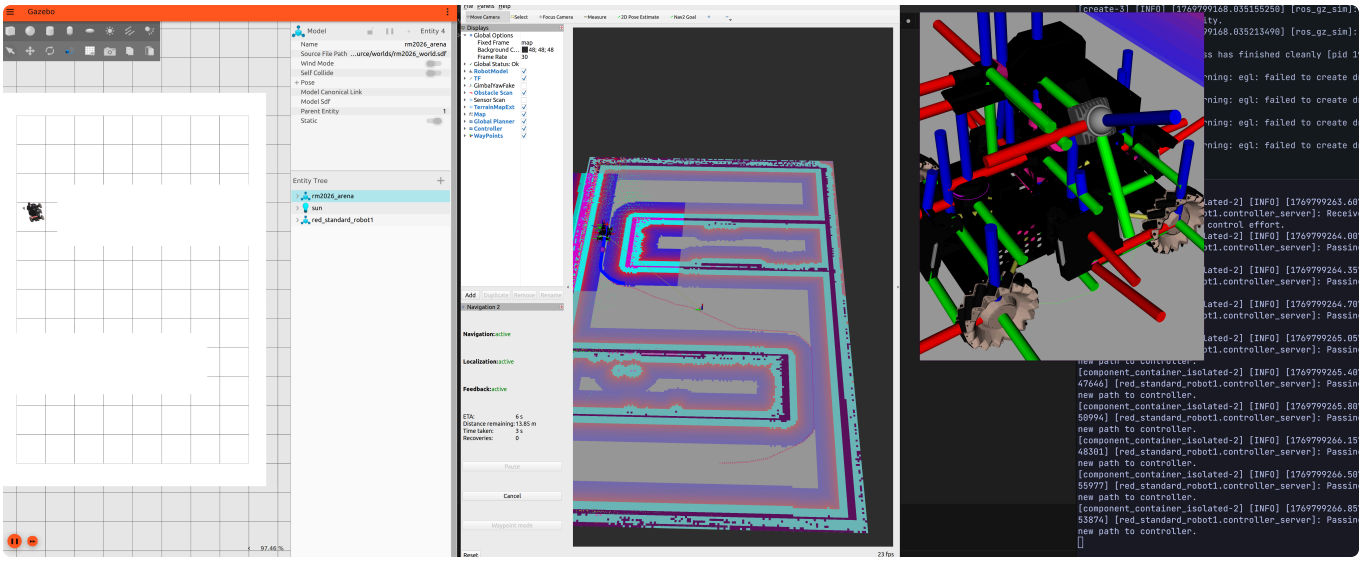
视觉组必备，高速旋转的装甲板小陀螺对于实际的弹丸打击是很头疼的事情，最终目标就是稳定的打击装甲板，来提高击杀效率，识别打击的目标包括：

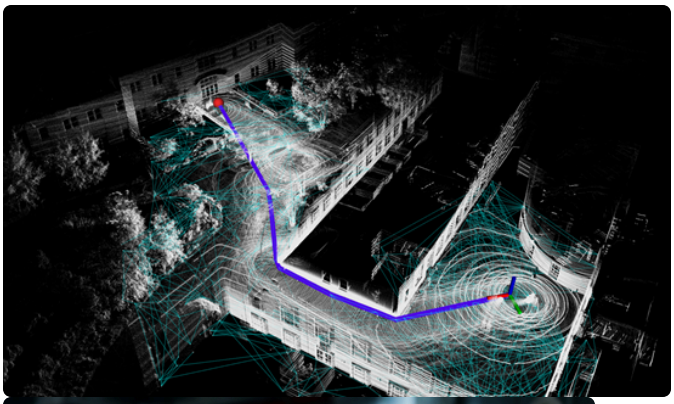
1. **前哨站与基地**
2. **地面车辆装甲板**
3. **能量机关**
   通过高效实时的识别，解算，追踪，规划，打击执行，通讯电控下位机的流程，确保精准的打击目标，涉及到感知，决策，规划，以及部分的控制

你会学到什么：

- OpenCV 与传统视觉：图像表示、颜色空间、阈值分割、轮廓、几何特征、候选筛选和图像可视化
- 神经网络与后端推理：目标检测、分类、关键点检测、模型输入输出、预处理、后处理、模型转换和推理后端，例如 ONNX Runtime、TensorRT、OpenVINO
- 相机模型与位姿解算：相机内参、畸变、目标几何、PnP、重投影和相机坐标系下的目标位姿
- 坐标变换与姿态表示：相机、云台、机体和世界坐标系之间的变换，旋转矩阵、欧拉角、四元数和变换链
- 目标跟踪与状态估计：目标关联、卡尔曼滤波、扩展卡尔曼滤波、漏检处理、运动预测和目标切换
- 运动模型与弹道解算：目标运动模型、云台运动模型、弹道模型、飞行时间、延迟补偿和预测瞄准
- 云台控制与设备通信：云台控制、开火判定、串口或 CAN 通信、命令时序、状态反馈和异常恢复

导航







哨兵机器人作为自主运行的机器人，无需操作手操控，需具备自主运行的能力，根据局势自主决策行为

不仅仅是“从A点到B点”，机器人需要根据自身状态、环境障碍、目标任务和底盘运动能力，持续生成可执行的运动，并在环境变化、定位异常、规划失败或通信中断时重新规划或安全停止，这就是导航

简单的来讲，一个能自主移动的机器人，本质上在做一件事，就是循环回答四个问题：

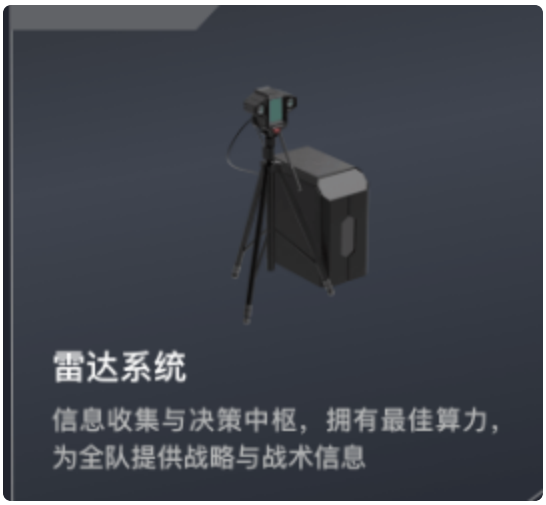
1. 我在哪里？ —— 定位与状态估计
2. 环境中哪里可以走？ —— 感知、地图与障碍建模
3. 我要完成什么任务？ —— 目标、任务状态和执行权限
4. 怎样安全地到达？ —— 路径、轨迹、控制和故障恢复
   对应一条数据流：
   传感器与里程计
   → 自身状态估计与定位
   → 环境模型与障碍表示
   任务目标
   → 路径规划
   → 连续轨迹与速度规划
   → 底盘控制
   → 实际运动反馈
   → 重规划、恢复或安全停止

你会学到什么：

- 机器人传感器与 TF2：激光雷达、IMU、轮速、里程计、坐标树、时间戳和传感器数据同步
- 点云处理与三维几何：点云滤波、降采样、地面分割、区域裁剪、特征提取、点云聚类和三维地图
- SLAM、LIO 与定位：同步定位与建图、激光惯性里程计、点云配准、回环、地图构建、重定位和地图匹配
- 多传感器状态估计：IMU、轮速、激光和其他位姿来源的融合，位姿、速度、偏差和定位质量估计
- 地图与环境建模：栅格地图、体素地图、占据地图、代价地图、距离场、地形地图、动态障碍和未知区域
- 路径规划与轨迹优化：图搜索、全局规划、局部规划、路径平滑、时间分配、轨迹优化和运动约束
- 底盘运动学与控制：底盘运动学、速度控制、路径跟踪、PID、MPC、QP、速度限制和执行器响应
- 任务规划与安全执行：状态机、行为树、任务目标、路径取消、任务抢占、重规划、故障恢复和安全停车

雷达





辅助兵种，架设高台之上，具备两个能力，识别与解析电磁波
识别部分：通过对整场车辆进行识别，持续跟踪并标记，达到高亮效果，施加双倍易伤效果
解析电磁波部分：通过解析、识别电磁波，为己方提供信息和优势

你会学到什么：

- OpenCV 与相机几何：图像处理、目标区域提取、相机内外参、地面投影、射线与平面求交以及场地坐标转换
- 神经网络与后端推理：整车检测、装甲板检测、目标分类、模型预处理、后处理、模型部署和推理性能优化
- 点云处理与三维感知：点云坐标变换、降采样、区域裁剪、近邻搜索、静态地图差分、动态点提取和聚类
- 激光定位与点云配准：点云地图、初始位姿、ICP/GICP 等配准方法、匹配质量和场地坐标系维护
- 动态目标检测：地面目标、高空目标、动态点云、空间聚类、目标质心和目标有效性判断
- 多目标跟踪与数据关联：轨迹创建与删除、卡尔曼滤波、目标关联、漏检预测、轨迹确认和身份维护
- 多传感器融合：相机语义结果与激光几何结果的时间对齐、坐标统一、空间匹配、颜色和编号融合
- 场地地图与态势可视化：场地坐标、区域语义、地图显示、目标轨迹显示、调试工具和状态监控
- 消息接口与系统通信：目标坐标、编号、颜色、有效状态、时间信息、哨兵接口和通信异常处理
- 电磁信号处理：信号采集、频谱分析、信号检测、特征提取、目标识别和通信接口

开发环境与基本操作

为什么需要开发环境?

在前面的导论中，我们已经了解了算法组的主要工作，但算法并不是写出一段代码就能够直接在机器人上运行

从代码编写到最终部署，一个实际的算法工程通常需要经过以下过程：

编写源代码
→ 编译与构建
→ 程序运行
→ 结果验证与调试
→ 修改与重新构建
→ 版本管理与团队协作
→ 实际设备部署

例如，一个能够识别装甲板的算法程序，不仅包含目标识别的计算逻辑，还需要能够读取相机图像、调用相应的依赖库、完成编译构建，并最终在指定的计算平台上运行
因此，我们首先需要认识几个在后续开发中经常接触的工具：

工具	主要作用
Ubuntu	提供程序开发和运行所需的操作系统环境
Terminal / Shell	通过命令与操作系统及其他程序交互
VS Code	代码编辑、工程管理和调试
GCC / G++	编译 C/C++ 源代码
CMake	描述和组织 C/C++ 项目的构建过程
Git	代码版本管理与团队协作
GitHub / GitLab	远程代码托管与协作

Ubuntu 与 Linux

什么是 Linux?

Linux 是一种开源的操作系统内核（Kernel），与 Windows 所采用的 NT 内核类似，它负责管理计算机的硬件资源，并通过系统调用接口为上层程序提供基础服务。基于 Linux 内核构建的完整操作系统，通常被称为 Linux 发行版（Linux Distribution）

比较常见的发行版包括：

- Ubuntu
- Debian

- Fedora
- Arch Linux
  不同发行版在软件包管理、默认配置和软件版本等方面存在差异，但都基于 Linux 内核构建
  Ubuntu 就是其中一种，也是机器人开发中经常使用的 Linux 发行版

为什么是 Ubuntu（Linux）？

对于算法组来说，我们不仅需要在自己的电脑上编写代码，还需要将程序部署到机器人搭载的计算平台上

Linux 在机器人开发、嵌入式设备、服务器以及开源软件工程中具有广泛应用。许多机器人相关的开发框架、设备驱动和工具也提供了相应的 Linux 支持，具备成熟的机器人软件生态、丰富的开发工具、便捷的硬件接口及良好的远程开发支持

而使用 Ubuntu 则主要因为团队协作成本和现有生态，队内统一使用**Ubuntu 22.04系统**

Ubuntu 环境的安装

目前常见的使用方式包括：

1. 双系统：在电脑中安装 Windows 和 Ubuntu，启动时选择需要使用的系统
2. 虚拟机：通过虚拟化软件，在现有操作系统中运行 Ubuntu
3. WSL2：在 Windows 中使用 Linux 环境

这里我们使用双系统以获得原生的体验，虚拟机和wsl的方案均有其局限性
具体的安装教程与注意事项自行搜索，自行制作安装u盘安装

终端、Shell与命令行

以下部分请在安装双系统后在Linux环境下阅读

启动Ubuntu后，如果没有进行任何修改，使用`Ctrl + Alt + T`打开默认终端

打开终端后，通常可以看到类似下面的内容：

```
username@ubuntu:~$
```

其中：

- `username`：当前用户名
- `ubuntu`：当前计算机的主机名
- `~`：当前所在目录，这里表示用户主目录

- `$`：普通用户 Shell 提示符
  不同终端和 Shell 配置的显示形式可能有所区别

终端能做什么？

```
sudo apt install cowsay fortune-mod
```

```
cowsay "hello itl"
```

```
curl wttr.in/Tianjin
```

```
curl -s 'wttr.in/Tianjin?format=3' | cowsay
```

```
python3 -m http.server 8000
```



什么是终端?

在平时使用电脑时，我们通常通过图形用户界面（GUI）与计算机交互，例如双击打开文件、使用文件管理器复制内容、通过软件窗口完成操作

而终端（Terminal）提供了另一种交互方式：通过输入命令，让计算机执行相应的操作

例如，我们可以通过文件管理器创建一个文件夹，也可以在终端中输入一条命令来完成同样的事情

Shell 又是什么?

Shell是运行在终端中的命令解释器，终端负责提供命令输入与输出的交互界面，而 Shell 负责解释用户输入的命令，并组织相应程序的执行

Ubuntu 中常见的 Shell 是 Bash

一个简单的交互过程可以表示为：
输入命令

→ 终端接收输入
→ Shell 解释命令
→ 执行相应程序
→ 程序输出结果
→ 终端显示结果

Linux 基本命令

目录操作

命令	功能
`pwd`	查看当前工作目录
`ls`	查看目录内容
`ls -a`	显示包含隐藏项在内的目录内容
`ls -l`	以详细格式显示目录内容
`cd`	切换工作目录
`mkdir`	创建目录

文件操作

命令	功能
`touch`	创建空文件，或更新已有文件的时间戳
`cat`	输出文件内容
`cp`	复制文件或目录
`mv`	移动或重命名文件
`rm`	删除文件
`rm -r`	递归删除目录及其内容

需要注意，`rm` 删除文件通常不会将文件移入桌面环境的回收站。
尤其是：

```
rm -rf <目录>
```

其中 `-r` 表示递归处理目录，`-f` 表示强制处理部分情况。这类命令具有较高的破坏性，执行前必须明确目标路径和操作范围

常用终端操作技巧

实际开发中，我们不需要每次都完整地手动输入所有命令
以下几个操作能够明显提高使用效率：

操作	功能
`Tab`	尝试自动补全命令或路径
`↑ / ↓`	浏览历史命令
`Ctrl + R`	反向搜索历史命令
`Ctrl + Shift + C`	在常见终端中复制
`Ctrl + Shift + V`	在常见终端中粘贴
`Ctrl + C`	向当前前台进程发送中断信号
`Ctrl + Z`	暂停当前前台进程

用户权限与软件管理

普通用户与 root

Linux 是一个多用户操作系统，不同用户对文件、目录和系统资源可以具有不同的访问权限

普通用户通常能够在自己拥有权限的目录中创建、修改和删除文件，但不能直接修改所有系统文件

root 则是具有特殊管理权限的用户

在 Ubuntu 中，我们经常通过 `sudo` 在经过授权后，以其他用户（默认通常为 root）的身份执行指定命令

例如：

```
sudo apt update
```

执行时需要输入当前用户的密码

APT 软件包管理

Ubuntu 使用 APT 进行软件包管理

常见命令包括：

```
# 更新软件包索引
sudo apt update

# 安装指定软件包
sudo apt install <软件包名称>

# 卸载指定软件包
sudo apt remove <软件包名称>
```

其中，`apt update` 主要用于更新软件包索引，并不会直接将系统中所有已经安装的软件升级到最新版本

`apt upgrade` 则用于根据当前配置的软件源及依赖关系升级已安装的软件包

VS Code

主流轻量级代码IDE**编辑器**，使用简便，页面简约，拥有丰富的插件与第三方支持，几乎支持所有主流语言
对于后续的 C++ 开发，我们通常会用到：

- 文件与工程目录管理
- C/C++ 代码编辑与语法提示
- 全局搜索
- 集成终端
- 断点与调试
- Git 版本差异查看

自行安装vscode后，在终端中进入某个项目目录后，可以输入：

```
code .
```

这里的 `.` 就是前面提到的当前目录

VS Code 集成终端与前面打开的系统终端一样，都可以执行相应的 Shell 命令

github

自行创建github帐号

最主流的代码托管仓库与管理平台，程序员的最大开源交流平台，上亿用户与开源代码托管仓库，几乎涵盖所有主流编程语言以及行业方向的优质开源，同时与Git结合，可以作为优秀的管理

工具，将自己的代码通过Git托管到Github个人仓库

Git 与版本控制

为什么需要版本控制?

在实际的算法开发中，我们通常需要不断修改程序

例如：

- 修改装甲板识别的筛选条件
- 调整状态估计算法的参数
- 修复某个数据处理错误
- 增加新的功能模块
- 多个队员同时开发同一套代码

但如果每次修改都直接覆盖原有文件，我们很难清楚地知道之前改过什么，也不方便在出现问题时恢复到某个已知版本

Git 就是为解决这类问题而广泛使用的分布式版本控制系统

它可以记录代码的修改历史，并支持多人在不同分支上进行开发与协作

需要注意，Git 和 GitHub 并不是同一个东西

Git 是版本控制工具，GitHub、GitLab 等则提供远程仓库托管及相关协作功能。即使完全不连接网络，我们也可以在本地使用 Git

Git 仓库

一个普通目录可以通过：

```
git init
```

初始化为 Git 仓库

执行后，目录中通常会生成一个隐藏的 `.git` 目录，用于保存仓库的版本管理信息

`git clone url` 会将远程仓库复制到本地，并完成相应的仓库初始化与远程配置，因此不需要再对同一仓库重复执行 `git init`

Git 的基本工作流程

需要先理解以下几个概念：

```
工作区 Working Directory
    |
    | git add
    v
暂存区 Staging Area
    |
    | git commit
    v
本地仓库 Local Repository
    |
    | git push
    v
远程仓库 Remote Repository


远端新变化
    ——fetch / pull——→ 本地获取并按规则整合
```

工作区是我们实际编辑文件的地方

暂存区则用于组织下一次提交所包含的修改

执行 `git commit` 后，这些被暂存的修改将形成一次本地提交，成为仓库历史的一部分

如果需要把本地提交同步到团队的远程仓库，则可以在权限及分支配置正确的情况下使用 `git push`

基本操作

进入一个已经初始化的 Git 仓库后：

```
# 查看当前仓库状态
git status

# 查看尚未暂存的修改
git diff

# 将指定文件加入暂存区
git add main.cpp

# 查看已经暂存的修改
git diff --cached

# 提交修改
git commit -m "Update main program"

# 查看提交历史
git log --oneline

# 移除暂存文件
git restore --staged filename
```

首次提交前，需要配置自己的 Git 提交身份，例如：

```
git config user.name "Your Name"

git config user.email "your_email@example.com"
```

以上两条命令不加 `--global` 时，默认只针对当前仓库配置

.gitignore

在 C++ 开发中，CMake 通常会产生大量中间文件和构建产物

这些文件大多可以通过源码重新生成，因此通常不需要提交到 Git 仓库

我们可以在项目根目录下创建一个 `.gitignore` 文件，例如：

```
build/
output/
```

这样就可以让 Git 忽略对应目录中尚未被跟踪的文件

需要注意，`.gitignore` 不会自动取消已经被 Git 跟踪的文件。实际项目中需要根据情况维护版本管理范围

Markdown与Obsidian

Markdown 是一种用纯文本写格式化文档的方式
本质上还是 .txt，但通过一些简单符号表达标题、列表、代码、链接、图片等结构

为什么要用markdown

`Markdown` 是技术协作里最常用的轻量文档格式

- 解决
  - 文档能不能长期保存
  - 别人能不能快速看懂
  - 命令能不能直接复制
  - 报错能不能被搜索
  - 修改记录能不能被 Git 清楚追踪
  - 文档能不能在 GitHub、Obsidian、VS Code 里通用
- 开发场景
  - 写 README
  - 写环境配置教程
  - 写调试记录
  - 写问题排查过程
  - 写招新资料
  - 写 GitHub 项目文档
  - 写 Obsidian 笔记
- 优势
  1. 本质是纯文本，任何编辑器都能打开
  2. 语法简单，几分钟就能上手
  3. 对 Git 友好，可以清楚看到每一行改动
  4. 对代码友好，命令、配置、报错都能原样保存
  5. 对平台友好，GitHub、Obsidian、VS Code 都能直接显示
  6. 适合写 README、环境配置、调试记录、问题复盘、学习笔记

Obsidian

一款Markdown 笔记软件，推荐去官网下载AppImage格式

总结

本讲作为算法组的导论性质课程，其中只是大致介绍了Linux、终端、git等常用开发环境，各位意向加入算法组的同学，请clone 仓库 https://github.com/wzzzou/RM-ITL-Algorithm-Training.git
此仓库后续更新每一讲的pdf讲义以及markdown原文，同时有优质资源和教程网址汇总
大家可以直接阅读 PDF，需要查看原文时，clone整个仓库，在 Obsidian 中将 讲义原文 文件夹作为仓库打开，图片及宽度设置就能正常显示
请各位同学自行选择资料并配合ai的使用，继续深入学习Linux、git、终端等内容

本讲没有需要提交的强制作业，下一讲的内容是C++、ROS2和工程构建，请各位同学从你看到这里的当天开始，每天学习C++基础语法